

Gestione degli errori

Gestione degli errori

Come evitare che il programma si blocchi quando qualcosa va storto.

Cosa succede quando si verifica un errore?

Ogni tanto il programma incontra una situazione che non riesce a gestire: l'utente scrive del testo quando ci si aspetta un numero, si cerca di dividere per zero, si prova ad aprire un file che non esiste.

In questi casi Python "lancia" un **errore** (chiamato tecnicamente *eccezione*) e, se non lo gestisci, il programma si blocca e mostra un messaggio di errore.

Esempi di errori comuni:

```
print(10 / 0)           # ZeroDivisionError – non si può dividere per
                        # zero
print("ciao" + 5)       # TypeError – non puoi sommare testo e numero
int("pippo")           # ValueError – "pippo" non è un numero
print(variabile_che_non_c_e) # NameError – variabile non definita

lista = [1, 2, 3]
print(lista[10])        # IndexError – la lista non ha la posizione 10
```

Il blocco `try / except`

Per evitare che il programma si blocchi, "avvolgi" il codice rischioso in un blocco `try`. Se si verifica un errore, Python salta al blocco `except` invece di fermarsi:

```
try:
    risultato = 10 / 0
except ZeroDivisionError:
    print("Non puoi dividere per zero!")
```

Traduzione in parole: "**Prova** a eseguire questo codice. Se si verifica un `ZeroDivisionError`, **invece** fai questo."

Gestire più tipi di errori

Puoi avere più blocchi `except`, uno per ogni tipo di errore che vuoi gestire:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
    print(f"10 / {numero} = {risultato}")
except ValueError:
    print("Devi inserire un numero valido, non del testo.")
except ZeroDivisionError:
    print("Non puoi dividere per zero.")
```

Il blocco `else` — "tutto è andato bene"

Il blocco `else` viene eseguito **solo se non si è verificato nessun errore**:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
except (ValueError, ZeroDivisionError) as e:
    print(f"Errore: {e}")
else:
    print(f"Risultato: {risultato}") # viene eseguito solo se tutto è ok
```

Il blocco `finally` — "sempre, in ogni caso"

Il blocco `finally` viene **sempre eseguito**, che ci sia stato un errore o no. Torna utile per fare "pulizia" (chiudere un file, liberare risorse):

```
try:
    file = open("esempio.txt", "r")
    contenuto = file.read()
except FileNotFoundError:
    print("File non trovato.")
finally:
    print("Operazione completata (con o senza errori).")
```

Leggere il messaggio di errore

Puoi salvare il messaggio di errore con `as` e stamparlo:

```
try:
    x = int("pippo")
except ValueError as e:
    print(f"Dettaglio errore: {e}")
# Dettaglio errore: invalid literal for int() with base 10: 'pippo'
```

Gli errori più comuni

Errore	Quando succede
<code>ValueError</code>	Valore sbagliato per il tipo (es. <code>int("abc")</code>)
<code>TypeError</code>	Tipo sbagliato (es. <code>"ciao" + 5</code>)
<code>ZeroDivisionError</code>	Divisione per zero
<code>NameError</code>	Variabile non definita
<code>IndexError</code>	Indice fuori dalla lista
<code>KeyError</code>	Chiave non trovata nel dizionario
<code>FileNotFoundError</code>	File non trovato

Esempio pratico: input sicuro

```
while True:
    try:
        a = float(input("Numeratore: "))
        b = float(input("Denominatore: "))
        risultato = a / b
    except ValueError:
        print("Inserisci solo numeri validi.")
        continue
    except ZeroDivisionError:
        print("Non si può dividere per zero.")
        continue
    else:
        print(f"Risultato: {risultato}")
        break
```